Model Question Paper - I (DDCO)

Module-1

1. (a) Demonstrate the positive and negative logic using AND gate.

Ans:→ Hardware digital gates are defined in terms of signal values such as H and L. For example, the electronic gate shown in fig: 1(ii). The truth table for this gate is shown in fig 1(i).

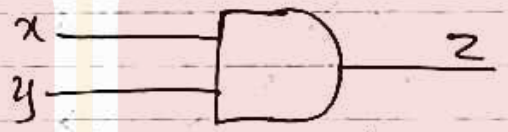
x	y	z
L	L	L
L	H	L
H	L	L
H	H	H

(a) Truth Table with H & L

(b) Gate block diagram

x	y	z
0	0	0
0	1	0
1	0	0
1	1	1

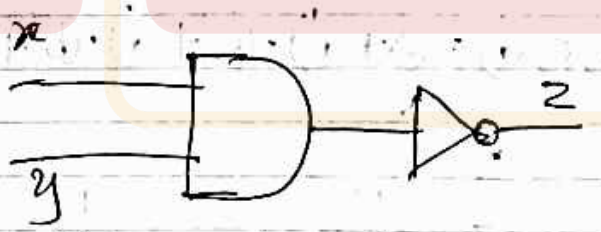
(c) Truth table for positive logic



(d) Positive logic AND gate

x	y	z
0	0	1
0	1	1
1	0	1
1	1	0

(e) Truth Table for negative logic



(f) Negative logic AND gate

Fig 1: Demonstration of positive and negative logic.

It specified the physical behavior of the gate when H is 3V and L is 0V.

The truth table of fig 1(c) assumes a positive logic assignment, with H=1 and L=0. This truth table is same as the one for the AND operation shown in fig 1(d).

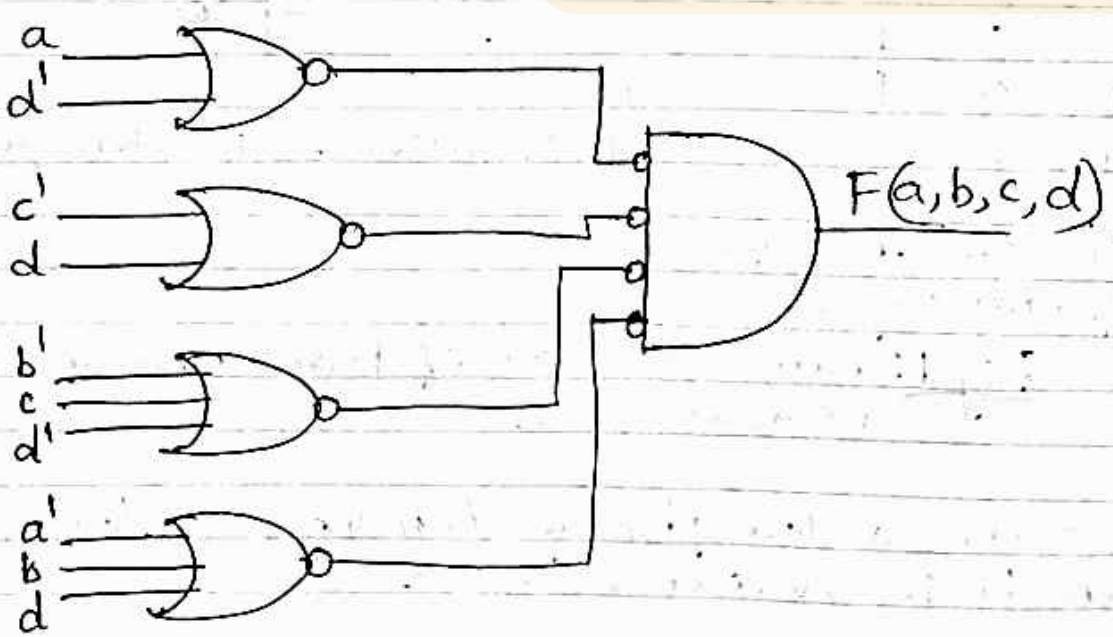
Now consider the negative logic assignment for the same gate with L=1 and H=0, truth table is shown in fig 1(e). The graphic symbol for negative logic AND gate is shown in fig 1(f).

1. (b) Find the POS expression for $F(a,b,c,d) = \Pi(2,3,5,8,10,13,14) + d(1,6,7,11)$ and realize it using NOR gates.

Ans:

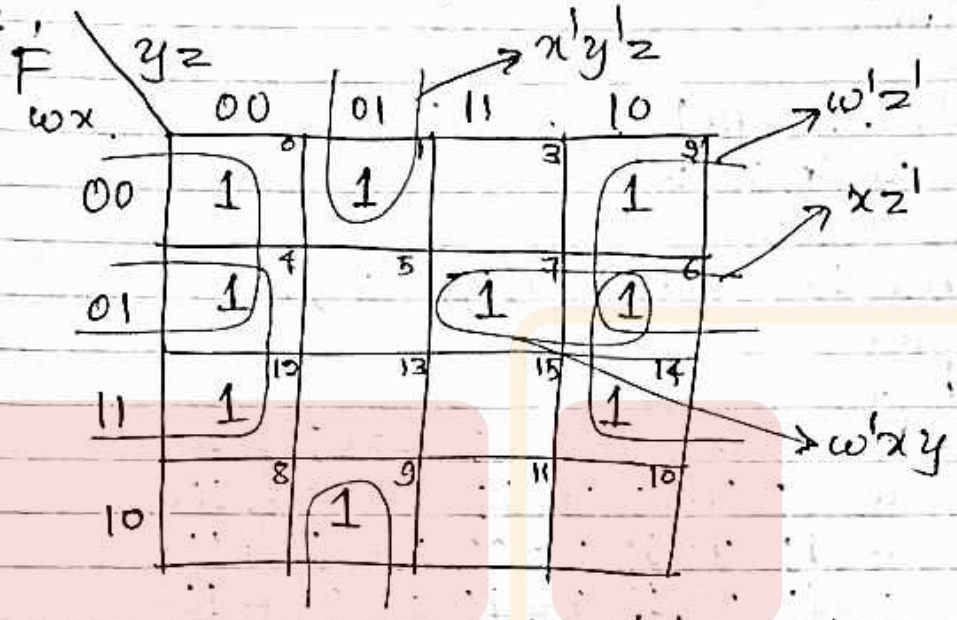
F (ab \ cd)	00	01	11	10	
00	0	X 1	0 3	0 2	$(a+d')$
01	0 4	X 5	X 7	X 6	$(c'+d)$
11	0 12	0 13	15	0 14	$(b'+c+d')$
10	0 8	X 9	11	0 10	$(a'+b+d)$

$F(a,b,c,d) = (a+d')\cdot(c'+d)\cdot(b'+c+d')\cdot(a'+b+d)$.

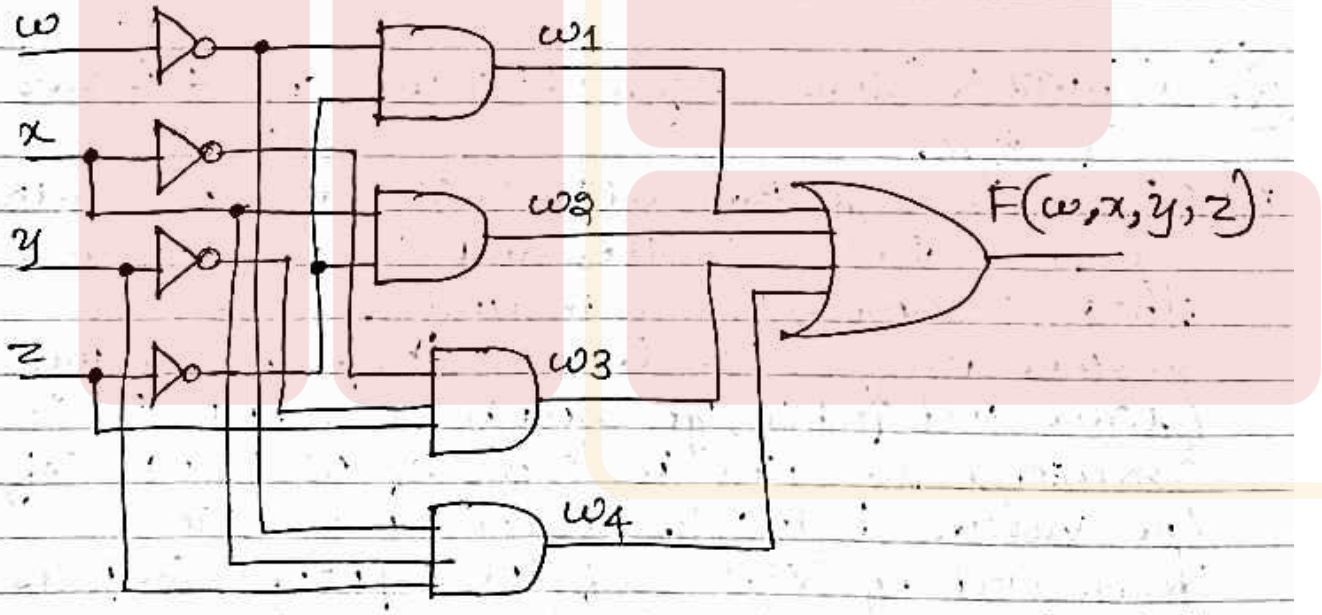


(1) (c) Simplify the Boolean function, $F(w,x,y,z) = \Sigma(0,1,2,4,6,7,9,12,14)$ using k-map and write the Verilog program for realizing the minimized expression.

Ans:



$F(w,x,y,z) = w'z' + xz' + x'y'z + w'xy$




```
module circuit123(f, w, x, y, z);
    input w, x, y, z;
    output f;
    wire w1, w2, w3, w4;
    wire wd, xd, yd, zd;
    not(wd, w);
    not(xd, x);
    not(yd, y);
    not(zd, z);
```

```
        and(w1, wd, zd);
        and(w2, x, zd);
        and(w3, xd, yd, z);
        and(w4, wd, x, y);
        or(f, w1, w2, w3, w4);
endmodule

module circuit123_tb();
    reg w, x, y, z;
    wire f;
    circuit123 uut(w, x, y, z, f);
    initial
      begin
        w=0; x=0; y=0; z=0; #100
        w=0; x=1; y=1; z=0; #100
        w=1; x=0; y=0; z=1; #100
        w=1; x=1; y=1; z=0; #100
      end
endmodule.
```

(2.) (a) What is Binary logic? List out any 4 Laws of logic.

Ans: Binary logic deals with variables that take on two discrete values and with operations that assume logical meaning. These two values may be called by different names (true and false, yes and no, etc.), but it is convenient to think in terms of bits and assign the values 1 and 0. The variables are designated by letters of the alphabet, such as, A, B, C, x, y, z, etc., with each variable having two and only two distinct possible values: 1 and 0.

Four Laws of Logic ⇒

Postulate 2	(a) $x+0=x$	(b) $x \cdot 1 = x$
Postulate 5	(a) $x+x'=1$	(b) $x \cdot x' = 0$
Theorem 1	(a) $x+x=x$	(b) $x \cdot x = x$
Theorem 2	(a) $x+1=1$	(b) $x \cdot 0 = 0$

Theorem 3, involution ⟶ $(x')' = x$

Theorem 4, associative (a) $x+(y+z)=(x+y)+z$ (b) $x(yz)=(xy)z$

(9) (b) Simplify the following Boolean function and find its SOP:

(i) $F(x,y,z) = \Sigma(0,1,4,5,6) + d(2,3,7)$

(ii) $F(w,x,y,z) = \Sigma(5,6,7,12,14,15) + d(13,9,11)$

Ans: (i)

F: x \ yz	00	01	11	10
0	1 (0)	1 (1)	X (3)	X (2)
1	1 (4)	1 (5)	X (7)	1 (6)

→ 1

So, simplified function and its SOP is,

$$F(x,y,z) = 1$$

(ii)

F: wx \ yz	00	01	11	10
00	0 (0)	0 (1)	0 (3)	0 (2)
01	0 (4)	1 (5)	1 (7)	1 (6)
11	1 (12)	X (13)	1 (15)	1 (14)
10	0 (8)	X (9)	X (11)	0 (10)

→ xz, → xy, → wx

So, simplified function and its SOP is,

$$F(w,x,y,z) = xz + xy + wx$$

(9.) (c) Write a short note on Hardware Description Language.

Ans: A hardware description language (HDL) is a computer-based language that describes the hardware of digital systems in a textual form. It describes hardware structures and the behavior logic circuits. HDL represents logic diagrams, truth tables, Boolean expressions, & complex abstractions of the behavior of a digital system.

It describes a relationship between signals that are the inputs to a circuit and the signals that are the outputs of the circuits.

HDL is a documentation language used in several major steps in the design flow of an integrated circuit: design entry, functional simulation or verification, logic synthesis, timing verification, and fault simulation.

Design entry creates an HDL-based description of the functionality that is to be implemented in hardware. Based on the HDL, the description can be in a variety of forms: Boolean logic equations, truth tables, a netlist of interconnected gates, or an abstract behavioral model.

Logic simulation displays the behavior of a digital system through the use of a computer. A simulator interprets the HDL description & either produces readable output like a time-ordered sequence of input and output signal values, or displays waveforms of the signals. The stimulus (i.e., the logic values of the inputs to a circuit) that tests the functionality of the design is called a test bench.

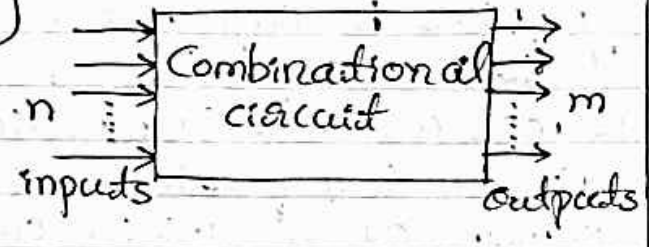
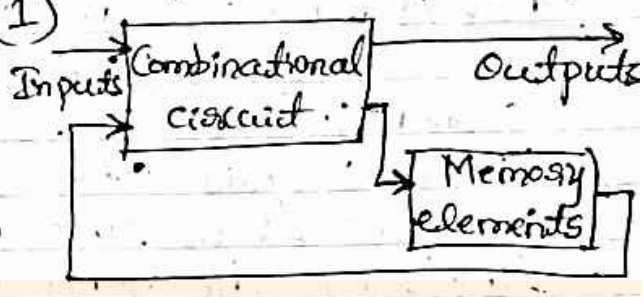
Logic synthesis is the process of deriving a list of physical components and their interconnections (called a netlist) from the model of a digital system described in an HDL. The netlist can be used to fabricate an integrated circuit or to lay out a printed circuit board with the hardware counterparts of the gates in the list.

Timing verification confirms that the fabricated, integrated circuit will operate at a specified speed. It checks each signal path to verify that it is not compromised by propagation delay.

Fault simulation compares the behavior of an ideal circuit with the behavior of a circuit that contains a process-induced flaw. It is used to identify input stimuli that can be used to reveal the difference between the faulty & fault-free circuits.

(3.)(a) Explain the differences between Combinational and sequential Circuits with their block diagrams and examples.

Ans:

Combinational	Sequential
(1) 	(1) 
(2) Generates output based on present inputs.	(2) Generates output based on present input as well as past output.
(3) Memory element is not present.	(3) Memory element is present.
(4) No feedback loop	(4) Feedback loop is present.
(5) Combination of operands not necessarily in same sequence.	(5) Sequence of operands is necessary in sequential circuit.
(6) Clock signal is not used.	(6) Clock signal is used to provide synchronization.
(7) Design process is much simple.	(7) Designing process is not simple.
(8) Examples: Half Adder, Full Adder, Encoder, Decoder, Multiplexer, Demultiplexer, etc.	(8) Examples: Flipflops, Registers, Counters, etc.

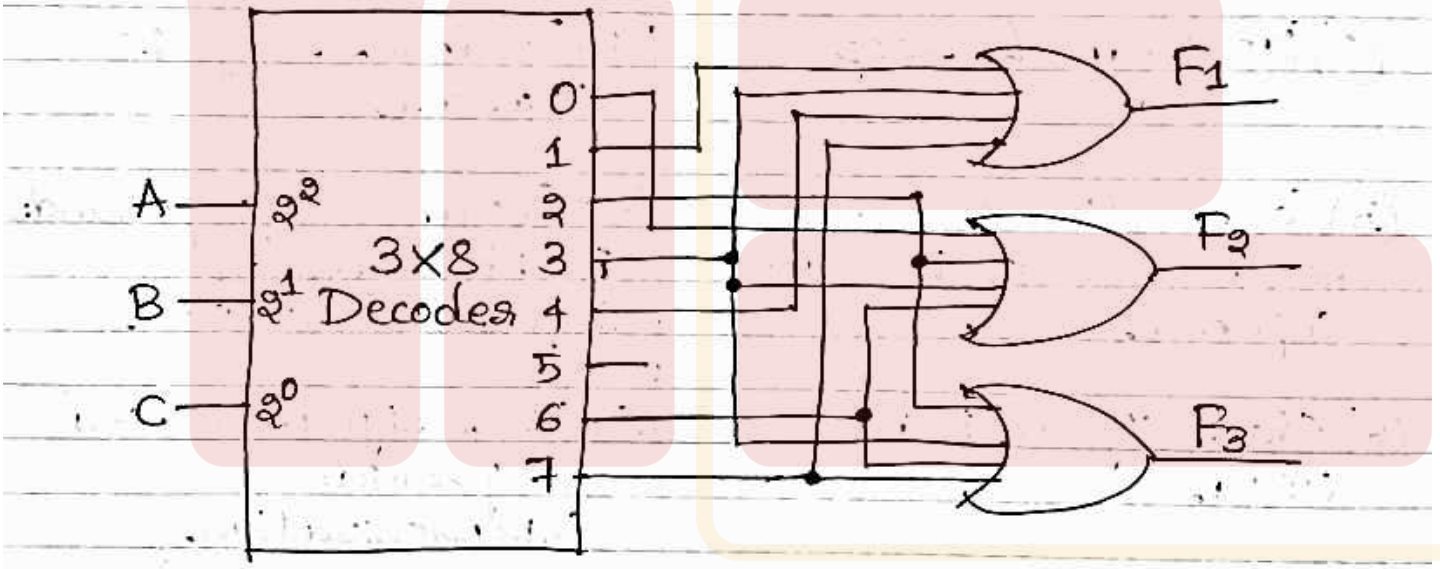
(3)(b) What are decoders? Implement the following Boolean functions with a decoder:

$F_1(A, B, C) = \Sigma m(1, 3, 4, 7)$
$F_2(A, B, C) = \Sigma m(0, 2, 3, 6)$ and
$F_3(A, B, C) = \Sigma m(2, 3, 6, 7)$

Ans: A binary code of n bits is capable of representing upto 2^n distinct elements of coded information. Decoders are the combinational circuits that converts binary information from n input lines to a maximum of 2^n unique output lines. If the n-bit coded information has unused combinations, the decoder may have fewer than 2^n outputs.

$F_1(A, B, C)$, $F_2(A, B, C)$, $F_3(A, B, C)$

These functions have three inputs and a total of eight minterms, a three-to-eight line decoder is needed. The implementation is,



(3)(c) What are Multiplexers? Implement the Boolean function $F(A, B, C, D) = \Sigma m(1, 3, 4, 11, 12, 13, 14, 15)$ with a 8:1 MUX.

Ans:

Multiplexers are the combinational circuits that selects binary information from one of many input lines and directs it to a single output line. The selection of a particular input line is controlled by a set of selection lines. Normally, there are 2^n input lines & n selection lines whose bit combinations determine input selected.

$F(A,B,C,D) = \Sigma m(1,3,4,11,12,13,14,15)$

This function is implemented with a multiplexer with three selection lines as shown in fig 3(c). First variable A must be connected to selection input S2 so that A, B, C correspond to selection inputs S2, S1, S0, respectively.

A	B	C	D	F	
0	0	0	0	0	F=D
0	0	0	1	1	
0	0	1	0	0	F=D
0	0	1	1	1	
0	1	0	0	1	F=D'
0	1	0	1	0	
0	1	1	0	0	F=0
0	1	1	1	0	
1	0	0	0	0	F=0
1	0	0	1	0	
1	0	1	0	0	F=D
1	0	1	1	1	
1	1	0	0	1	F=1
1	1	0	1	1	
1	1	1	0	1	F=1
1	1	1	1	1	

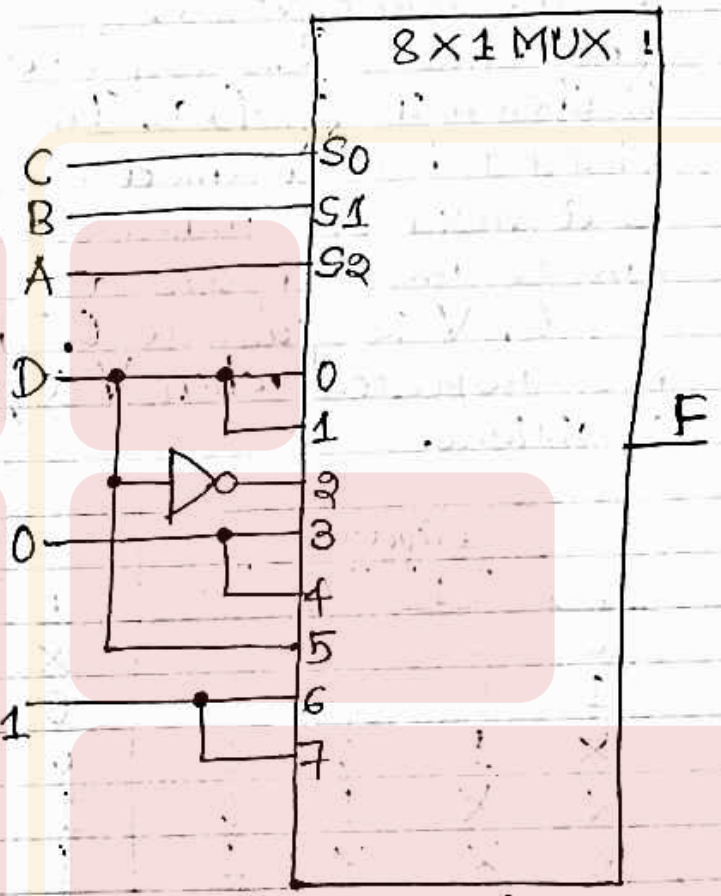


Fig 3(c): Implementing a four-input function with a multiplexer

The values for the data inputs are determined from the truth table listed in the figure. The corresponding data line number is determined from the binary combination of ABC. For example, the table shows that when ABC = 101, F = D, so the input variable D is applied to data input 5. The binary constants 0 and 1 correspond to two fixed signal values. When integrated circuits are used, logic 0 corresponds to signal ground and logic 1 is equivalent to the power signal, depending on the technology (e.g. 3V).

(4.) (a) Define encoder. Design a Four-input Priority Encoder.

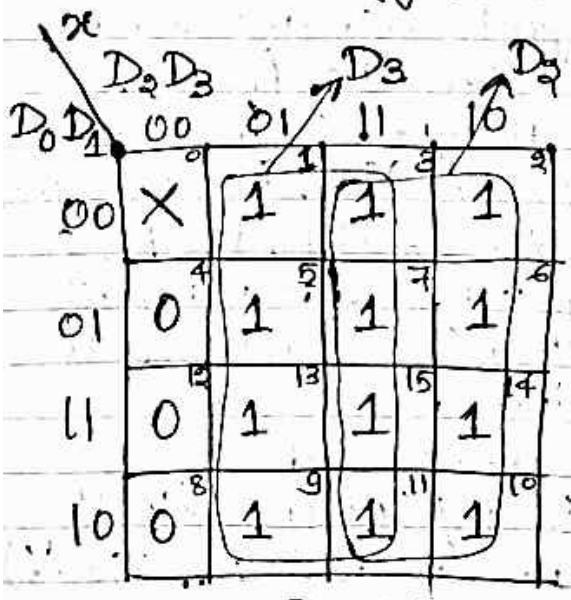
Ans: An encoder is a digital circuit that performs the inverse operation of a decoder. An encoder has 2^n (or fewer) input lines and n output lines. The output lines as an aggregate, generate the binary code corresponding to the input value.

Four-input Priority Encoder → The truth table is shown in fig(4)(a). In addition to x, y, the circuit has a third output denoted by V; it is a valid bit indicator that is set to 1 when one or more inputs are equal to 1. If all inputs are 0, V is equal to 0. The other two outputs not inspected when V equals 0 are don't care conditions.

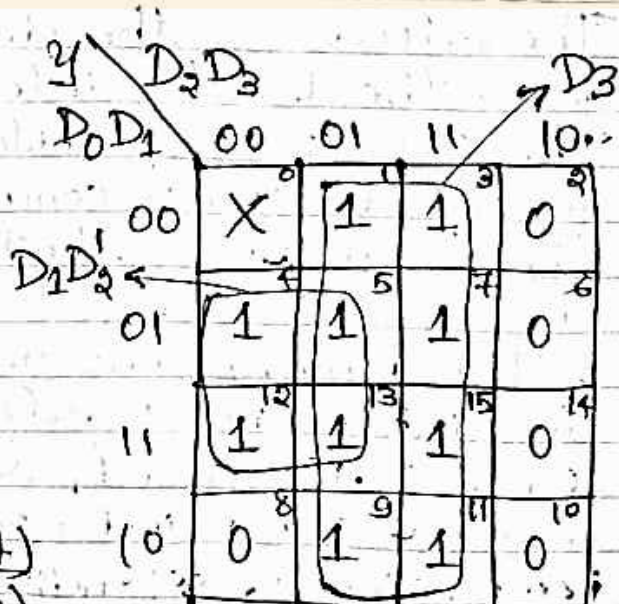
Inputs				Outputs		
D_0	D_1	D_2	D_3	x	y	V
0	0	0	0	X	X	0
1	0	0	0	0	0	1
X	1	0	0	0	1	1
X	X	1	0	1	0	1
X	X	X	1	1	1	1

Fig(4)(a)

Maps for simplifying outputs x and y are shown in fig(4)(b)

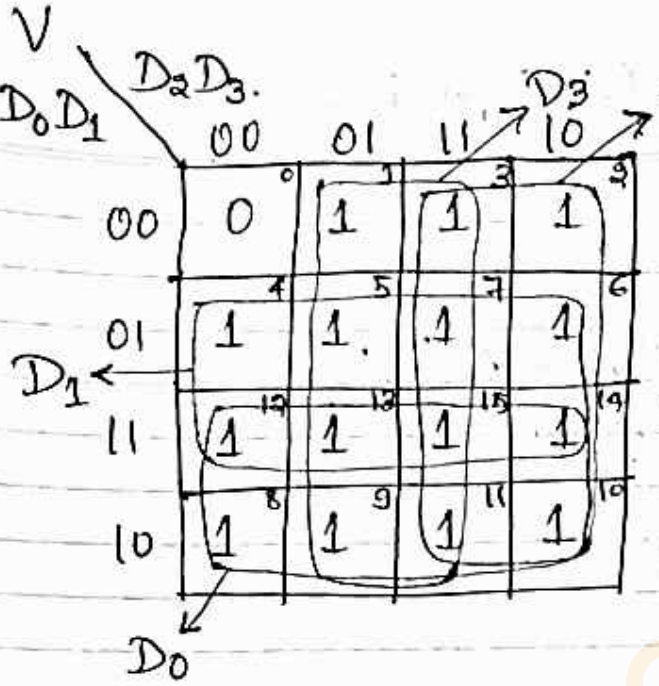


$x = D_2 + D_3$



$y = D_3 + D_1D_2'$

Fig: (4) (b)

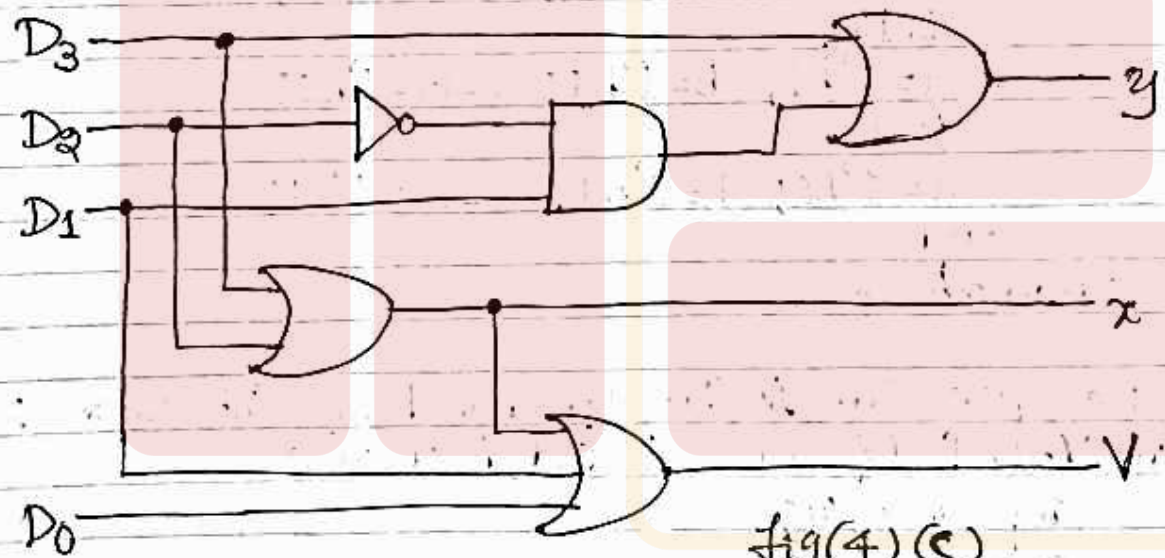


$V = D_0 + D_1 + D_2 + D_3$

fig:(4)(b)

The priority encoder is implemented in fig(4)(c) according to the following Boolean functions:

$$x = D_2 + D_3$$
$$y = D_3 + D_1 D_2'$$
$$V = D_0 + D_1 + D_2 + D_3$$



fig(4)(c)

(4)(b) Write the Verilog program to Implement Fuld Adder and Subtractor Circuits.

Ans: Verilog program to Implement Fuld Adder ⇒

```
module halfadder(s, c, a, b);
    input a, b;
    output s, c;
    xor(s, a, b);
    and(c, a, b);
endmodule
```

```
module FullAdder(S, Co, A, B, Ci);
    input A, B, Ci;
    output S, Co;
    wire w1, w2, w3;

    halfadder h1(w1, w2, A, B);
    halfadder h2(S, w3, w1, Ci);
    or(Co, w2, w3);
endmodule
module tb_fa();
    reg a, b, c_in;
    wire s, c_out;
    FullAdder F1(s, c_out, a, b, c_in);
     initial
       begin
        a=1'b0; b=1'b0; c_in=1'b0;
        #100
        a=1'b1; b=1'b0; c_in=1'b0;
        #100
        a=1'b1; b=1'b1; c_in=1'b1;
        #100
        a=1'b0; b=1'b1; c_in=1'b1;
      end
endmodule
```

Verilog program to Implement Full Subtractor ⇒

```
module halfsubtractor(d, b, x, y);
    input x, y;
    output d, b;
    xor(d, x, y);
    and(b, ~x, y);
endmodule
module FullSubtractor(D, Bo, X, Y, Bi);
    input X, Y, Bi;
    output D, Bo;
    wire w1, w2, w3;

    halfsubtractor h1(w1, w2, X, Y);
    halfsubtractor h2(D, w3, w1, Bi);
    or(Bo, w2, w3);
endmodule
```

```
module tb_fs();
  reg x, y, b_in;
  wire d, b_out;
  FullSubtractor F1(d, b_out, x, y, b_in);
   initial
     begin
       x=1'b0; y=1'b0; b_in=1'b0;
       #100
       x=1'b1; y=1'b0; b_in=1'b0;
       #100
       x=1'b1; y=1'b1; b_in=1'b1;
       #100
       x=1'b0; y=1'b1; b_in=1'b1;
     end
endmodule
```

4.(c) Write the Characteristic table and Equations of SR, JK, D and T Flip Flops.

Ans:

SR Flip Flop: Characteristic Table is,

S	R	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	X
1	1	1	X

∴ Characteristic equation of SR Flip Flop is,

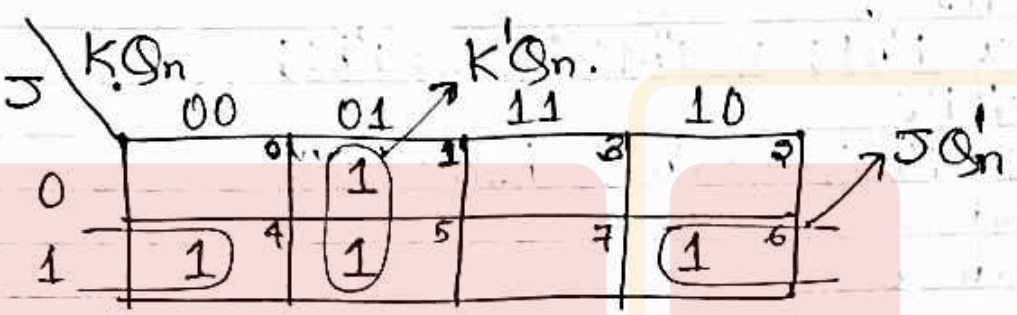
$$Q_{n+1} = S + R'Q_n$$

S \ RQ_n	00	01	11	10
0		1		
1	1	1	X	X

($R'Q_n$ → 01, S → 10)

JK Flip Flop: The characteristic Table of JK FF is shown below,

J	K	Q_n	Q_{n+1}	State
0	0	0	0	Q_n (Hold)
0	0	1	1	
0	1	0	0	Reset
0	1	1	0	
1	0	0	1	Set
1	0	1	1	
1	1	0	1	Toggle
1	1	1	0	



∴ Characteristic equation of JK FF is,

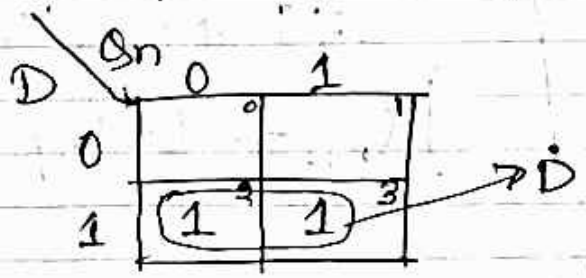
$$Q_{n+1} = JQ_n' + K'Q_n.$$

D Flip Flop: Characteristic Table of D FF is,

D	Q_n	Q_{n+1}
0	0	0
0	1	0
1	0	1
1	1	1

So, the characteristic equation of D FF is,

$$Q_{n+1} = D$$



T Flip Flop: Characteristic Table is shown below,

CLK	T	Q_n	Q_{n+1}
↓	0	0	0
↑	0	1	1
↑	1	0	1
↑	1	1	0

So, Characteristic equation,

$$Q_{n+1} = TQ_n' + T'Q_n$$

$$\Rightarrow Q_{n+1} = T \oplus Q_n$$

5. (a) What do you mean by an Addressing Mode? Explain any 5 Addressing Modes.

Ans: Programs are written in a high-level language. This enables the programmer to use constants, local variables, global variables, pointers and arrays. The compiler must be able to implement these constructs using the facilities provided in the instruction set of the computer in which the program will be run. The different ways in which the location of an operand is specified in an instruction are referred to as addressing modes.

Register mode ⇒ The operand is the contents of a processor register; the name (address) of the register is given in the instruction.

The instruction

Move LOC, R2

uses register mode. Processor registers are used as temporary storage locations where the data in a register are accessed using the Register mode.

Immediate mode ⇒ The operand is given explicitly in the instruction.

For example, the instruction

Move 200, immediate, R0

places the value 200 in register R0.

Absolute mode ⇒ The operand is in a memory location; the address of this location is given explicitly in the instruction.

The instruction

Move LOC, R2

uses absolute mode. It can represent global variables in a program. The declaration like

Integer A, B;

in a high-level language will cause the compiler to allocate a memory location to each of the variables A and B.

Autoincrement mode — The effective address of the operand is the contents of a register specified in the instruction. After accessing the operand, the contents of this register are automatically

incremented to point to the next item in a list. This mode is written as,

(Ri)+

The increment is 1 for byte-sized operands, 2 for 16-bit operands, and 4 for 32-bit operands.

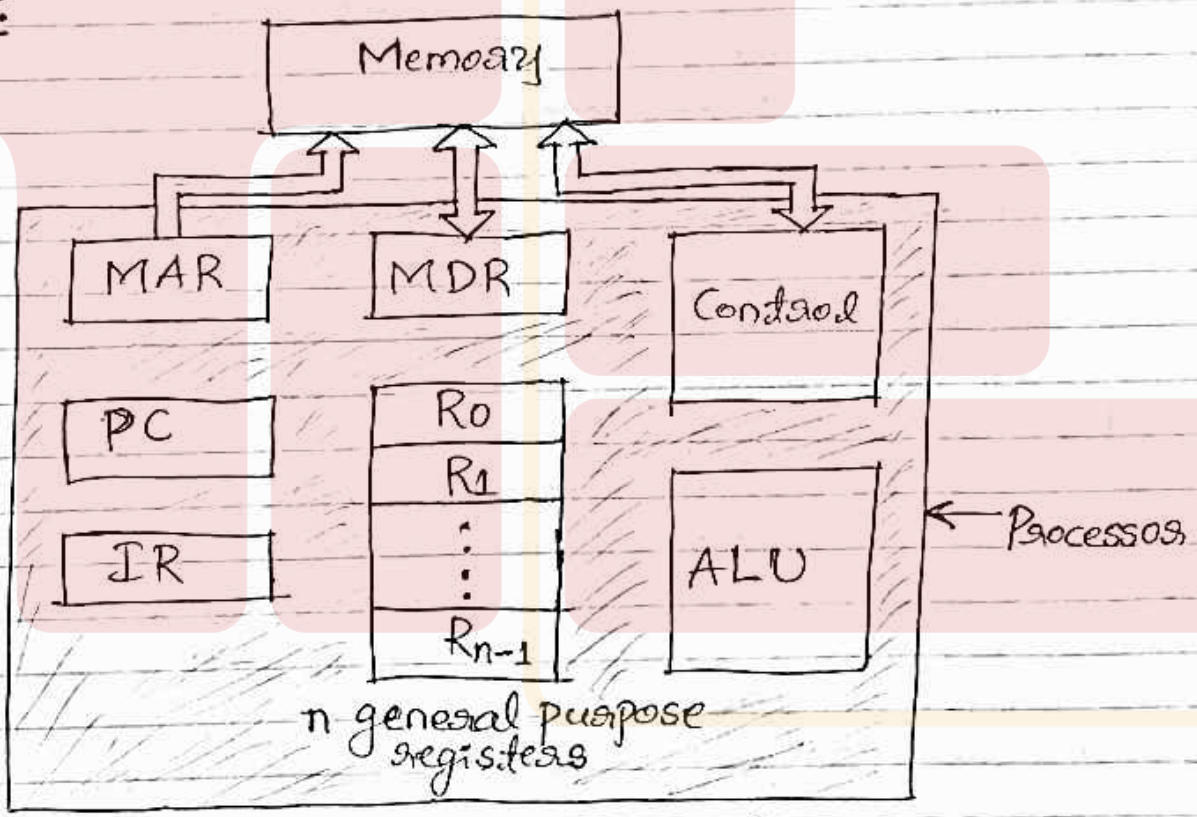
Autodecrement mode — The contents of a register specified in the instruction are first automatically decremented and are then used as the effective address of the operand.

This is written as,

−(Ri)

(5.)(b) Describe the functionality of the following: MAR, PC, IR, MDR and ALU

Ans:



MAR: (Memory Address Register) is a register facilitating communication with the memory. It holds the address of the location to be accessed. It holds the address contained in PC, also holds the address of the location where the result of the operation is stored.

PC: (Program Counter) is a specialized register to keep track of the execution of a program. It contains the memory address of the next instruction to be fetched and executed.

During the execution of an instruction, the PC points to the next instruction that is to be fetched from the memory.

IR: (Instruction Register) holds the instruction that is currently being executed. Its output is available to the control circuits, which generates the timing signals that control the various processing elements involved in executing the instruction. IR holds the contents of the MDR.

MDR: (Memory Data Register) contains the data to be written into or read out of the addressed location. It holds the operand that has been read from the memory. MDR holds the addressed word that is read out of the memory after a Read control signal.

ALU: (Arithmetic Logic Unit) performs the arithmetic operations. If an instruction involves an operation to be performed by the ALU, it is necessary to obtain the required operands. After the operands are fetched, the ALU can perform the desired operation.

(5.)(c) Explain Basic Performance equation and SPEC rating.

Ans: Basic Performance Equation:

Let T be the processor time required to execute a program that has been prepared in some high-level language.

Assume that complete execution of the program requires the execution of N machine language instructions. N is the actual number of instructions, not necessarily equal to the number of machine instructions in the object program.

Suppose that the average number of basic steps needed to execute one machine instruction is S, each basic step is completed in one clock cycle.

If the clock rate is R cycles per second, the program execution time is given by

$$T = \frac{N \times S}{R}$$

This equation is often referred to as the basic performance equation.
To achieve high performance, the value of T should be reduced. To do this, N and S should be reduced and R is increased.
SPEC Rating:- is computed as follows

$$\text{SPEC rating} = \frac{\text{Running time on the reference computer}}{\text{Running time on the computer under test}}$$

Thus a SPEC rating of 50 means that the computer under test is 50 times as fast as the UltraSPARC10 for this particular benchmark.
Let $SPEC_i$ be the rating for program i in the suite. The overall SPEC rating for the computer is given by

$$\text{SPEC rating} = \left(\prod_{i=1}^{n} SPEC_i \right)^{1/n}$$

where n is the number of programs in the suite. SPEC rating is a measure of the combined effect of all factors affecting performance, including the compiler, the operating system, the processor, and the memory of the computer being tested.

(6.)(a) Demonstrate the Branching operations using a loop to add n numbers with block diagram.

Ans: The task of adding a list of n numbers uses a separate Add instruction to add each number to the contents of Register R0. After adding all the numbers, the result is placed in memory location SUM. The addresses of the memory locations containing the n numbers are symbolically given as NUM1, NUM2, ..., NUMn. This method uses a long list of Add instructions.

Instead of this method, it is possible to place a single Add instruction in a program loop, as shown in fig (6)(a).

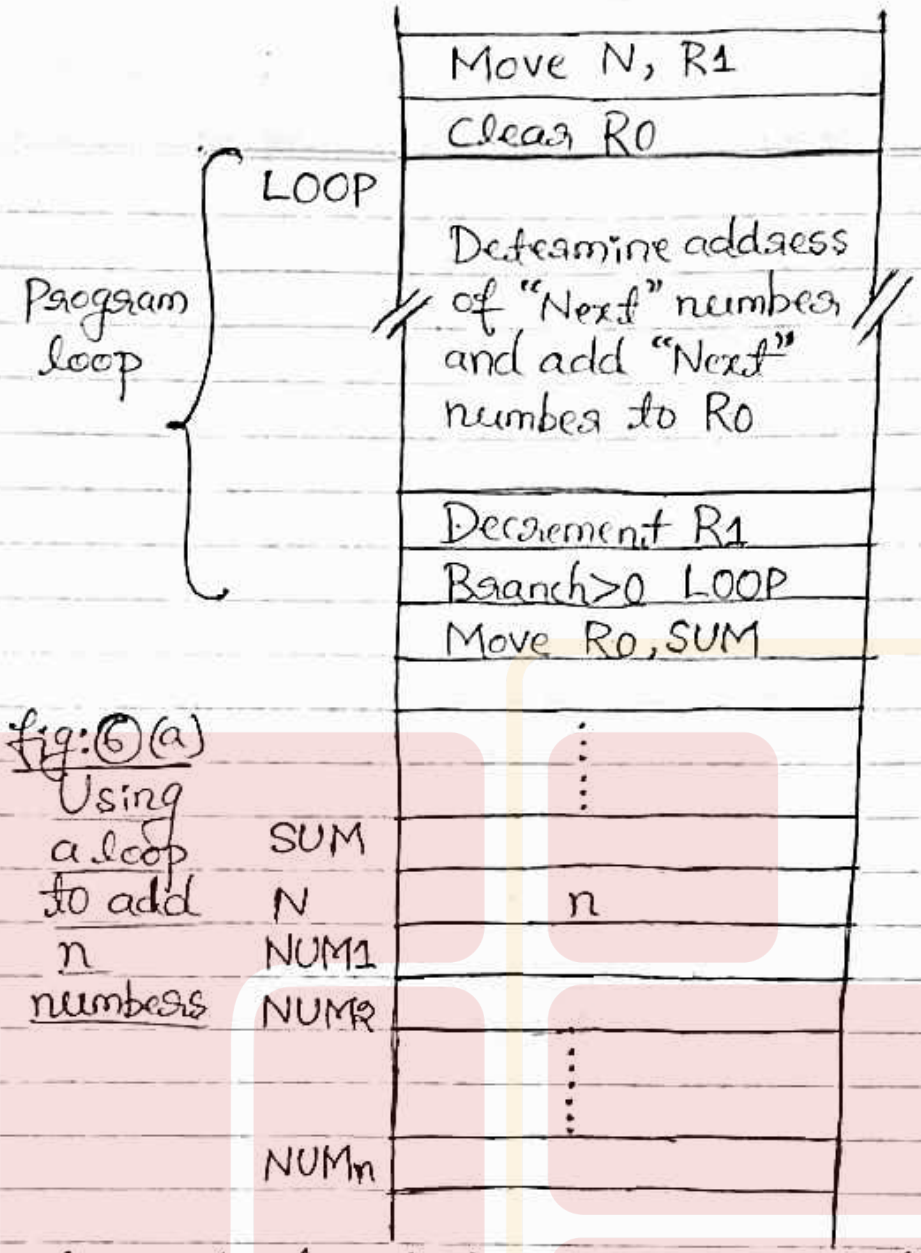


fig:⑥(a) Using a loop to add n numbers

The loop starts at location LOOP and ends at the instruction Branch>0. During each pass through this loop, the address of the next list entry is determined, and that entry is fetched and added to R0.

Assume that the number of entries in the list, n, is stored in memory location N, as shown. Register R1 is used as a counter to determine the number of times the loop is executed. So, the contents of location N are loaded into register R1 at the beginning of the program. Then, within the body of the loop the instruction

Decrement R1

reduces the contents of R1 by 1 each time through the loop.

In the program of fig⑥(a), the instruction

Branch>0 LOOP

(branch if greater than 0) is a conditional branch instruction that causes a branch to location LOOP if the result of the immediately preceding instruction, which is the decremented value in register R1, is greater than zero. At the end of the nth pass through the loop, the Decrement instruction produces a value of zero, and, hence branching doesn't occur. But, the Move instruction is fetched and executed. It moves the final result from R0 into memory location SUM.

(6.)(b) The Registers R1 and R2 has decimal values 1200 and 4600. Calculate the effective address of the memory operand in each of the following instructions when they are executed in sequence.

(i) Load 20(R1), R5 — effective address → EA
(ii) Move #3000, R5
(iii) Store R5, 30(R1, R2)
(iv) Add -(R2), R5
(v) Subtract (R1)+, R5

Ans:

(i) EA = [R1] + Offset — $R_1 = 1200$, Offset = 20
= 1200 + 20
EA = 1220

(ii) **EA = 3000**, because instruction uses immediate value 3000 only.

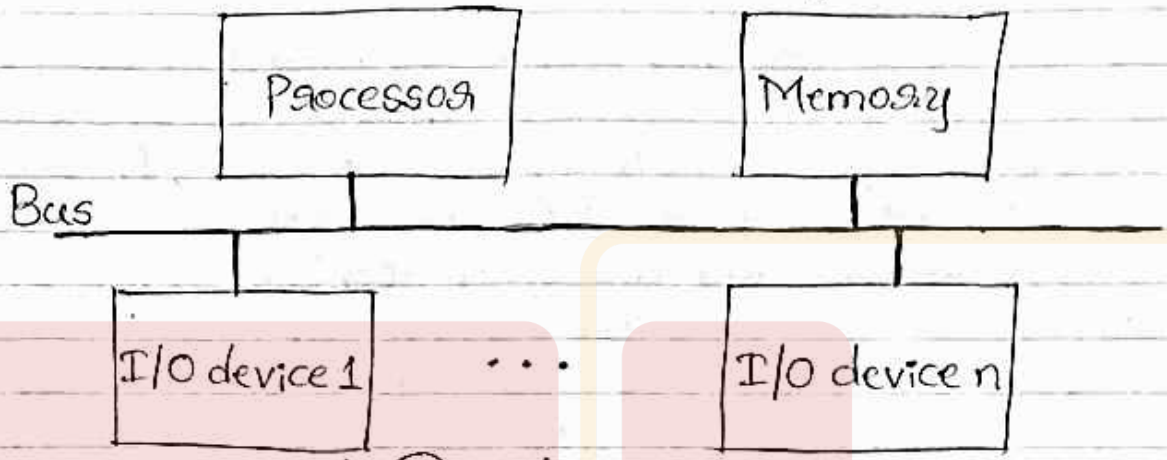
(iii) EA = [R1] + [R2] + Offset — $R_1 = 1200$, $R_2 = 4600$
= 1200 + 4600 + 30
EA = 5830

(iv) EA = [R2] − 1 — $R_2 = 4600$
= 4600 − 1
EA = 4599

(v) EA = [R1], since R1 = 1200.
⇒ **EA = 1200**

⑥.(c) Explain Single Bus Structure.

Ans: A simple arrangement to connect I/O devices to a computer is to use a single bus arrangement as shown in fig⑥(c). The bus enables all the devices connected to it to exchange information.



fig⑥(c) A single-bus structure

Typically, it consists of three sets of lines used to carry address, data, and control signals. Each I/O device is assigned a unique set of addresses. When the processor places a particular address on the address lines, the device that recognizes this address responds to the commands issued on the control lines. The processor requests either a read or a write operation, and the requested data are transferred over the data lines.

⑦.(a) Explain Memory mapped I/O and I/O interface for an input device with a diagram.

Ans:

When I/O devices and the memory share the same address space, the arrangement is called memory-mapped I/O.

With this, any machine instruction that can access memory can be used to transfer data to or from an I/O device.

For example, if DATAIN is the address of the input buffer associated with the keyboard, the instruction

'Move DATAIN, R0
reads the data from DATAIN and stores them into processor register R0. Similarly, the instruction

Move R0, DATAOUT

sends the contents of register R0 to location DATAOUT, which may be the output data buffer of a display unit or a printer.

Most computer systems use memory-mapped I/O. Some processors have special In and Out instructions to perform I/O transfers. For example, Intel processors have special I/O instructions and a separate 16-bit address space for I/O devices. When building a computer system based on these processors, the designer can connect I/O devices to use the special I/O address space or simply incorporate them as part of the memory address space. Here, the second approach is the most common as it leads to simpler software. One advantage of a separate I/O address space is that I/O devices deal with fewer address lines. A special signal on the bus indicates that the requested read or write transfer is an I/O operation. When this signal is asserted, the memory unit ignores the requested transfer. The I/O devices examine the low-order bits of the address bus to determine whether they should respond.

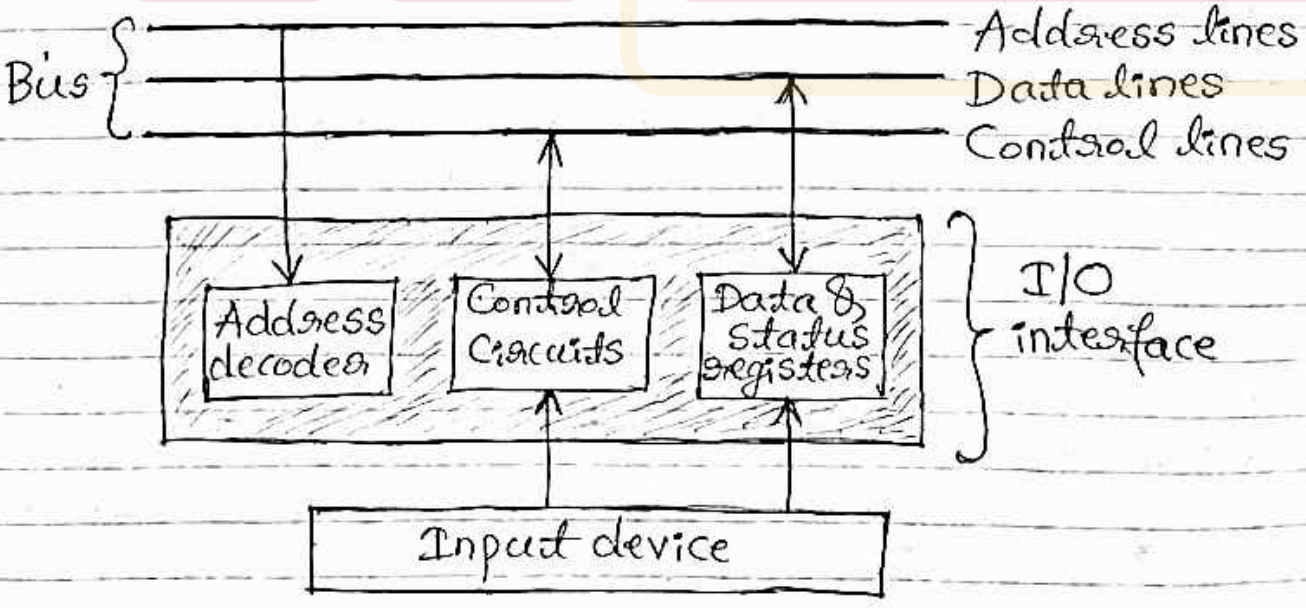


fig (7)(a) - I/O interface for an input device

Fig (7)(a) illustrates the hardware required to connect an I/O device to the bus.

→ The address decoder enables the device to recognize its address when this address appears on the address lines.
→ The data register holds the data being transferred to or from the processor.
→ The status register contains information relevant to the operation of the I/O device.
→ The data and status registers are connected to the data bus and assigned unique addresses.
→ The address decoder, the data and status registers, and the control circuitry required to coordinate I/O transfers constitute the device's interface circuit.

I/O devices operate at speeds that are vastly different from that of the processor. When the characters are entered at a keyboard, the processor is capable of executing millions of instructions between successive character entries. An instruction that reads a character from the keyboard should be executed only when a character is available in the input buffer of the keyboard interface. Also, an input character must be read only once.

(7.)(b) Explain I/O operations involving a keyboard and display device with a program that reads one line from keyboard, stores it in buffer and echoes it back to display.

Ans:

The four registers used in the data transfer operations are shown in fig (7)(b)(i). Register STATUS contains two control flags, SIN and SOUT, which provide status information for the keyboard and the display unit respectively. The two flags KIRQ and DIRQ in this register are used in conjunction with interrupts. The KEN and DEN bits are used with CONTROL register.

The data from the keyboard are made available in the DATAIN register, and data sent to the display are stored in the DATAOUT register.

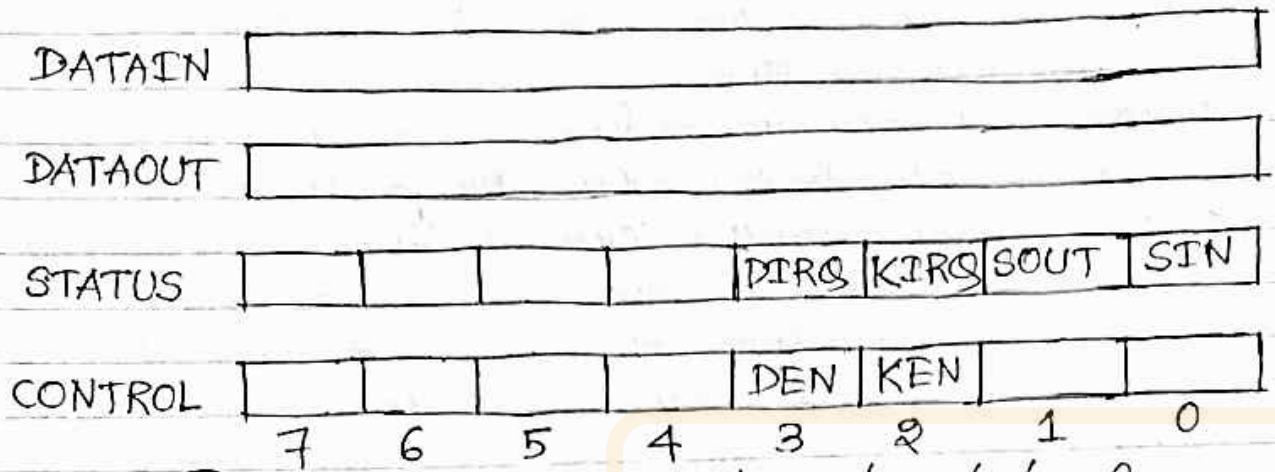


Fig (7)(b)(i) Registers in keyboard and display interfaces

```
        Move      #LINE, R0       Initialize memory pointer.
WAITK   TestBit   #0, STATUS      Test SIN.
        Branch=0  WAITK           Wait for character to be entered.
        Move      DATAIN, R1      Read character.
WAITD   TestBit   #1, STATUS      Test SOUT.
        Branch=0  WAITD           Wait for display to become ready.
        Move      R1, DATAOUT     Send character to display.
        Move      R1, (R0)+       Store character and advance pointer.
        Compare   #$0D, R1        Check if Carriage Return.
        Branch≠0  WAITK           If not, get another character.
        Move      #$0A, DATAOUT   Otherwise, send Line Feed.
        Call      PROCESS         Call a subroutine to process the input line.
```

Fig (7)(b)(ii) A program that reads one line from the keyboard, stores it in memory buffer, and echoes it back to the display

The program in fig (7)(b)(ii) reads a line of characters from the keyboard and stores it in a memory buffer starting at location LINE. Then, it calls a subroutine PROCESS to process the input line. As each character is read, it is echoed back to the display. Register R0 is used as a pointer to the memory buffer area. The contents of R0 are updated using the Autoincrement addressing mode

so that successive characters are stored in successive memory locations.
Each character is checked to see if it is the Carriage Return (CR) character, which has the ASCII code 0D (hex). If it is, a Line Feed character (ASCII code 0A) is sent to move the cursor one line down on the display and subroutine PROCESS is called. Otherwise, the program loops back to wait for another character from the keyboard.
This example illustrates program-controlled I/O in which the processor repeatedly checks a status flag to achieve the required synchronization between the processor and an input or output device.

(8.) (a) Explain how to handle interrupt from multiple devices using daisy chain and priority scheme.

Ans:
In case of simultaneous arrivals of interrupt requests from two or more devices, the processor must decide which request to service first. If several devices share one interrupt-request line, the schemes needed in this case are
(1) Daisy chain scheme.
(2) Priority scheme.

(1) In polling the status registers of the I/O devices, the priority is determined by the order in which the devices are polled. When vectored interrupts are used, only one device is selected to send its interrupt vector code. A widely used scheme is to connect the devices to form a daisy chain, as shown in fig (8)(a)(i). The interrupt-request line $\overline{INTR}$ is common to all devices. The interrupt-acknowledge line, INTA, is connected in a daisy-chain fashion. Here, the INTA signal propagates serially through the devices. When several devices raise an interrupt request and the $\overline{INTR}$ line is activated, the processor responds by setting the INTA line to 1. This signal is received by device 1. Device 1 passes the signal on to device 2 only if only if it does not require any service.

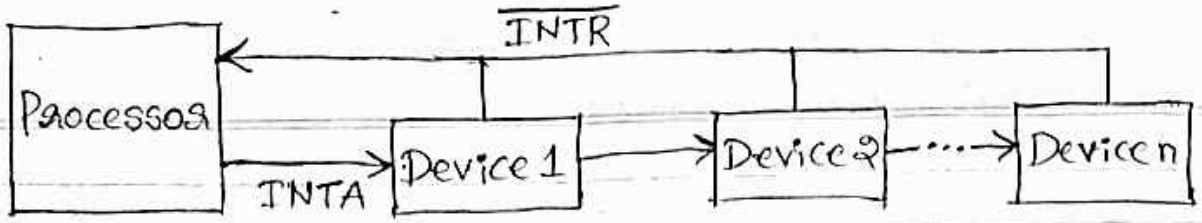
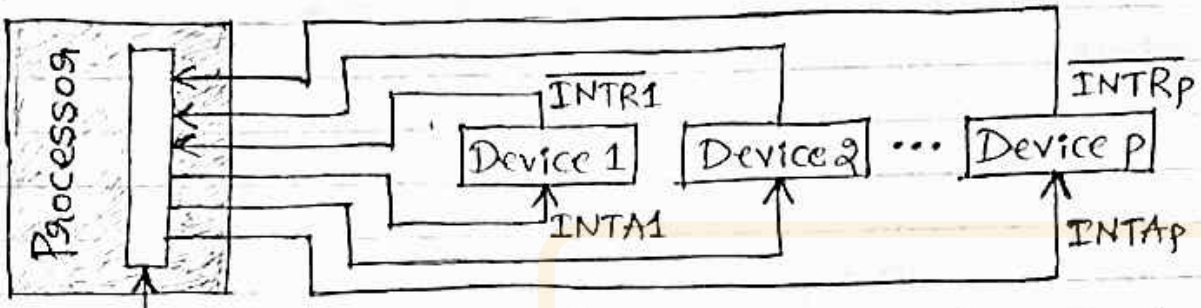
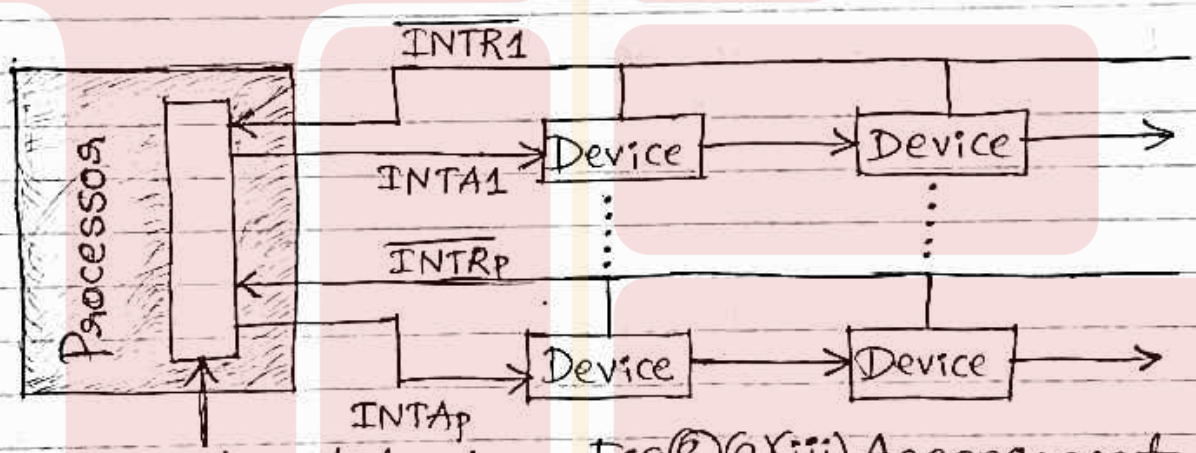


Fig (8)(a)(i) Daisy chain



Priority arbitration circuit

Fig (8)(a)(ii) Interrupt priority using individual interrupt-request and acknowledge lines



Priority arbitration circuit

Fig (8)(a)(iii) Arrangement of priority groups

If device 1 has a pending request for interrupt, it blocks the INTA signal and proceeds to put its identifying code on the data lines. So, in the daisy-chain scheme, the device that is electrically closest to the processor has the highest priority. The second device along the chain has second highest priority, and so on. The scheme of fig (8)(a)(i) requires considerably fewer wires than the individual connections in fig (8)(a)(ii).

(2) The main advantage of the multiple-priority scheme of fig (8)(a)(ii) is that it allows the processor

to accept interrupt requests from some devices but not from others, depending upon their priorities. The two schemes can be combined to produce the more general structure in fig (8)(a)(iii). Devices are organized in groups, and each group is connected at a different priority level. Within a group, devices are connected in a daisy chain. This organization is used in many computer systems.

(8)(b) Explain Centralized and Distributed Bus Arbitration approaches.

Ans:

Centralized Arbitration:

A basic arrangement in which the processor contains the bus arbitration circuitry is shown in figure fig (8)(b)(i). The processor is normally the bus master unless it grants bus mastership to one of the DMA controllers. A DMA controller needs

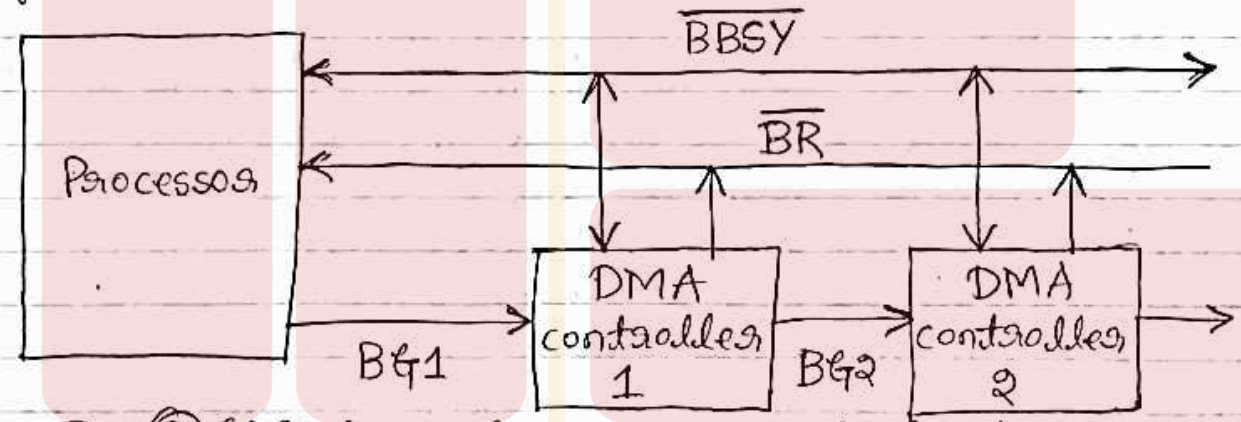


Fig: (8)(b)(i) A simple arrangement for bus arbitration using a daisy chain

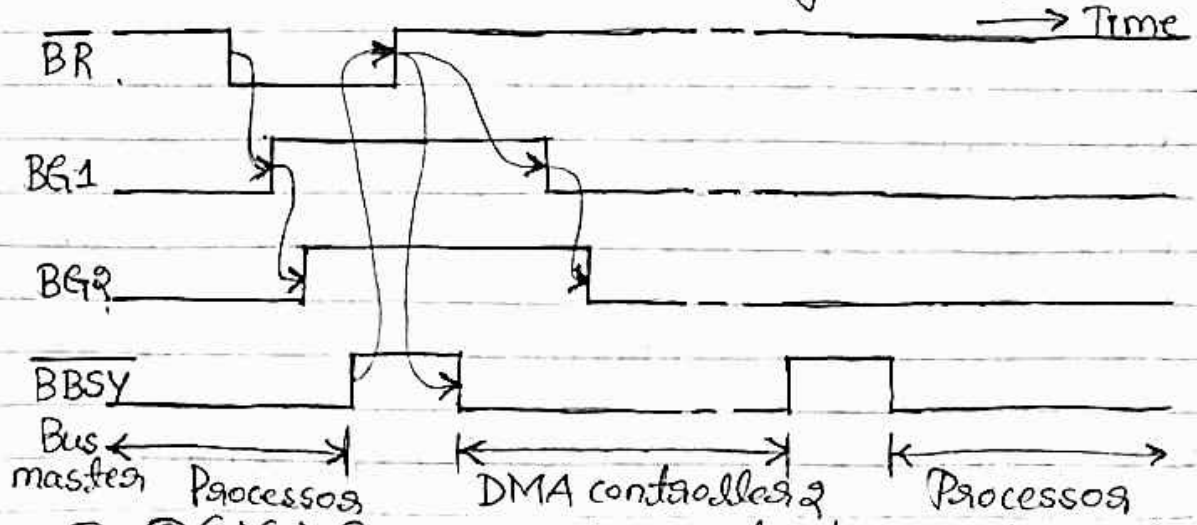


Fig (8)(b)(ii) Sequence of signals during transfer of bus mastership for the devices in fig (8)(b)(i)

to become the bus master by activating the Bus-Request line, $\overline{BR}$. The signal on the Bus-Request line is the logical OR of the bus requests from all the devices connected to it. When the Bus-Request is activated, the processor activates the Bus-Grant signal, BG1, indicating to the DMA controllers that they may use the bus when it becomes free. This signal is connected to all DMA controllers using a daisy-chain fashion. So, if DMA controller 1 is requesting the bus, it blocks the propagation of the grant signal to other devices. The current bus master indicates to all devices that it is using the bus by activating another open-collector line called Bus-Busy, $\overline{BBSY}$. So, after receiving the Bus-Grant signal, a DMA controller waits for Bus-Busy to become inactive, then assumes bus mastership of the bus. At this time, it activates Bus-Busy to prevent other devices from using the bus.

The timing diagram in fig (8)(b)(ii) shows the sequence of events for the devices in fig (8)(b)(i) as DMA controller 2 requests and acquires bus mastership and later releases the bus.

Figure (8)(b)(i) shows one bus-request line and one bus-grant line forming a daisy chain. Several such pairs may be provided in an arrangement of priority groups. This arrangement leads to considerable flexibility in determining the order in which requests from different devices are serviced. This ensures that only one request is granted at any given time, according to a predefined priority scheme. For example, if there are four bus request lines, BR1 through BR4, a fixed priority scheme may be used in which BR1 is given top priority and BR4 is given lowest priority. A rotating priority scheme may be used to give all devices an equal chance of being serviced, i.e., after a request on line BR1 is granted, the priority order becomes 2, 3, 4, 1.

Distributed Arbitration: means that all devices

waiting to use the bus have equal responsibility in carrying out the arbitration process, without using a central arbiter. A simple method for this is illustrated in fig (8)(b)(iii). Each device on the bus is assigned a 4-bit identification number. When one or more devices request the bus, they assert the $\overline{\text{Start-Arbitration}}$ signal and place their 4-bit ID numbers on four open-collector lines, $\overline{\text{ARB0}}$ through $\overline{\text{ARB3}}$. A winner is selected as a result of the interaction among the signals transmitted over these lines. The net outcome is that the code on the four lines represents the request that has the highest ID number.

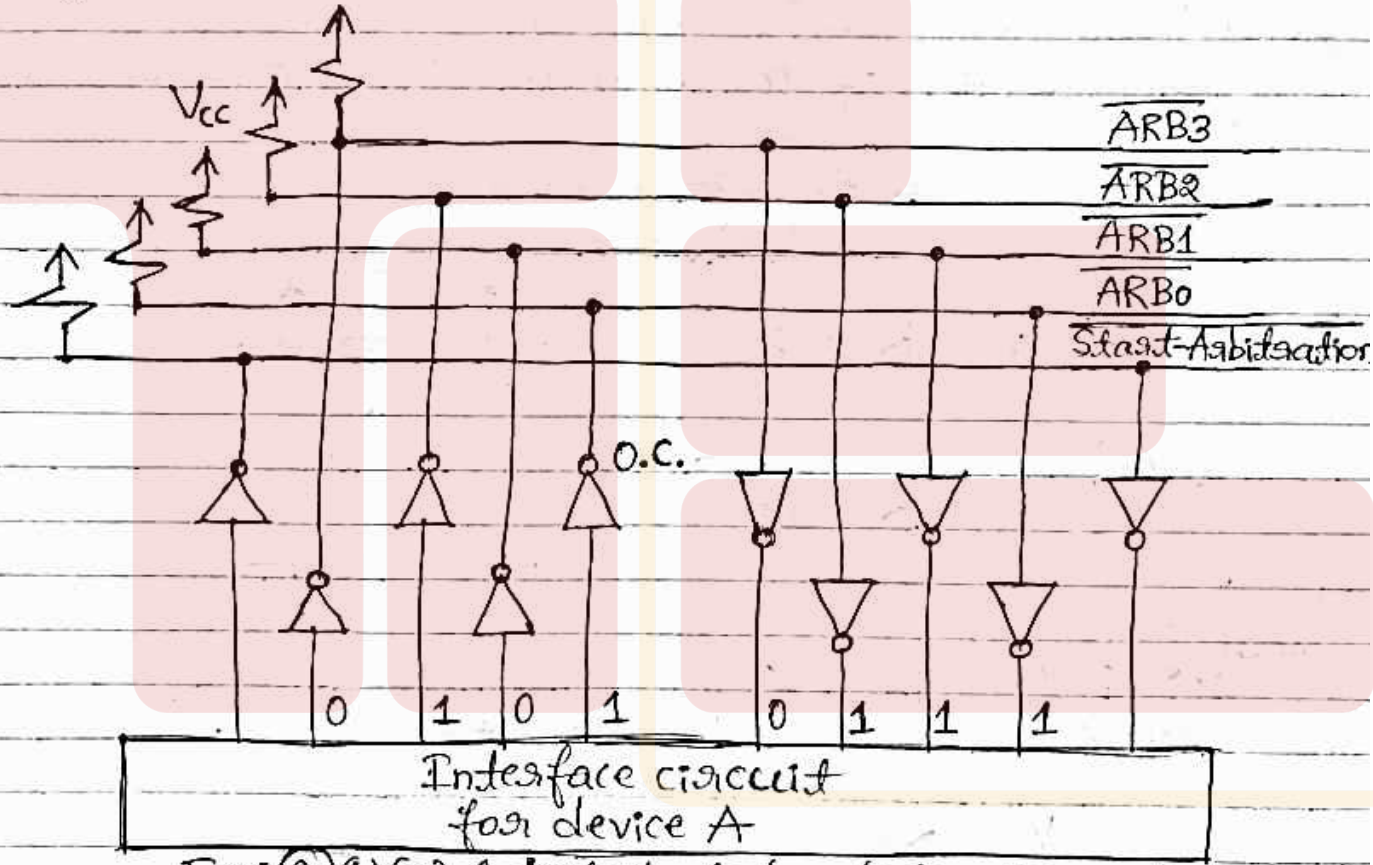


Fig: (8)(b)(iii) A distributed arbitration scheme

The drivers are of the open-collector type, the bus will be in the low-voltage state. Assume that two devices, A and B, having ID numbers 5 and 6, respectively, are requesting the use of the bus. Device A transmits 0101, & device B transmits 0110. The code seen by both devices is 0111. Each device compares the pattern on the arbitration lines to its own ID, starting from the MSB. If it detects a difference at any bit position, it disables its drivers

at that position and for all lower-order bits. It does so by placing a 0 at the input of these drivers. In the current example, device A detects a difference on line $\overline{ARB1}$. Hence, it disables its drivers on lines $\overline{ARB1}$ and $\overline{ARB0}$. This causes the pattern on the arbitration lines to change to 0110, which means that B has won the contention. Since the code on the priority lines is 0111 for a short period, device B may temporarily disable its drivers on line $\overline{ARB0}$. However, it will enable this drivers again once it sees a 0 on line $\overline{ARB1}$ resulting from the action by device A.

(9.) (a) With a diagram, explain the single bus organization of the data path inside a processor.

Ans:

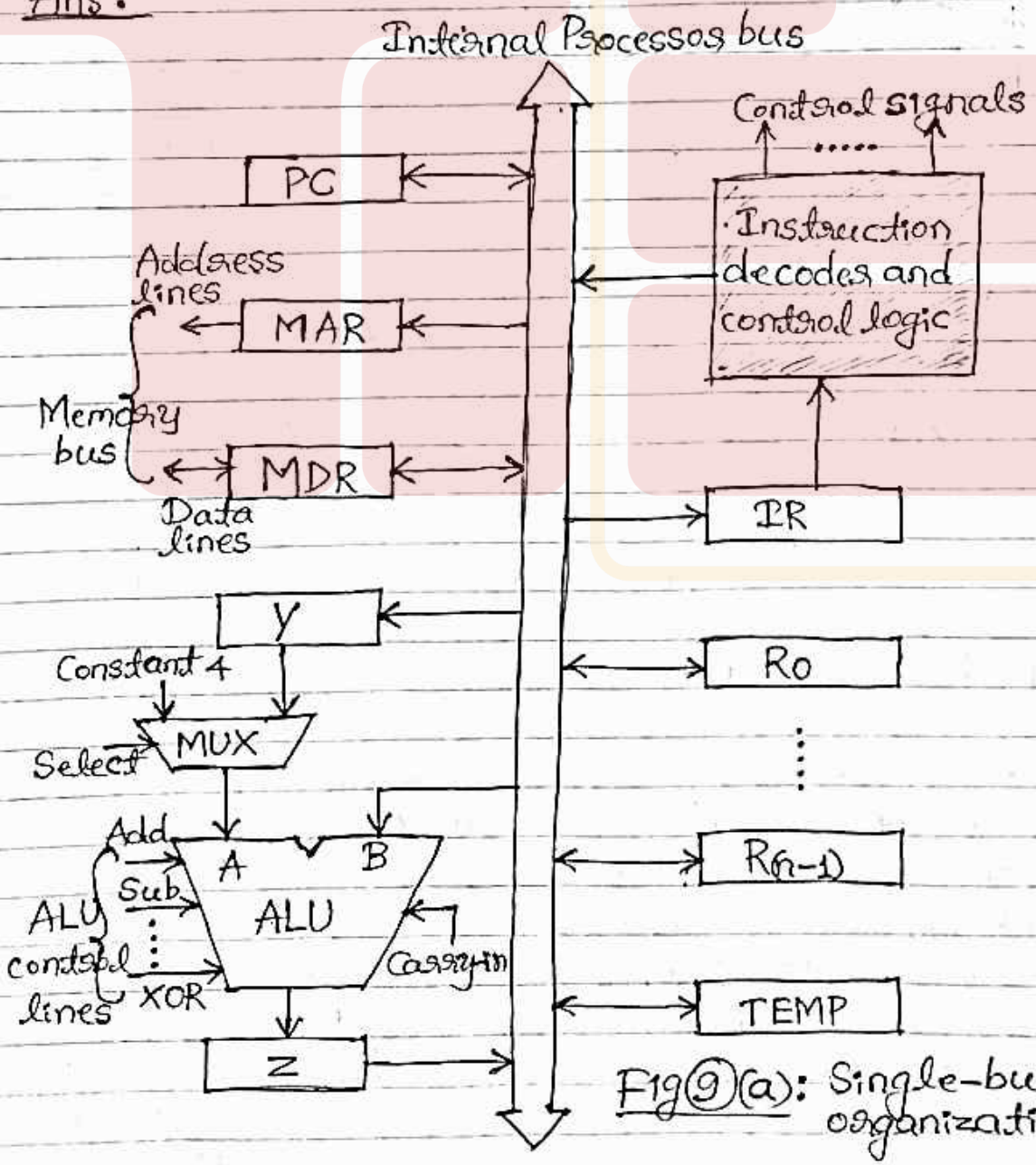


Fig (9)(a): Single-bus organization

- A single bus organization in which the arithmetic and logic unit (ALU) and all the registers are interconnected via a single common bus is shown in fig (9)(a). This bus is internal to the processor.

→ The data and address lines of the external memory bus are shown in fig (9)(a) connected to the internal processor bus via the memory data register, MDR, and the memory address register, MAR, respectively.

→ MDR has two inputs and two outputs. Data may be loaded into MDR either from the memory bus or from the internal processor bus. The data stored in MDR may be placed on either bus.

→ The input of MAR is connected to the internal bus, and its output is connected to the external bus.

→ The control lines of the memory bus are connected to the instruction decoder and control logic block. This unit is responsible for issuing the signals that control the operation of all the units inside the processor and for interacting with the memory bus.

→ The number and use of the processor registers R0 through R(n-1) vary considerably from one processor to another. Registers may be general-purpose or special-purpose registers such as index registers or stack pointers.

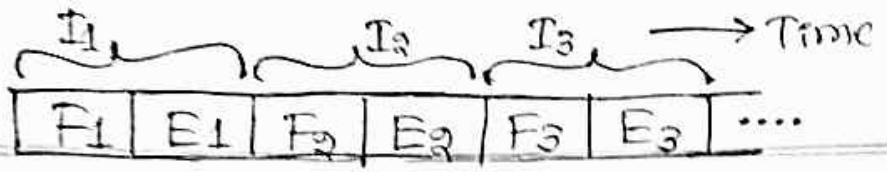
→ Three registers, Y, Z and TEMP in fig (9)(a) are transparent, i.e., the programmer need not be concerned with them because they are never referenced explicitly by any instruction. They are used for temporary storage during execution of some instructions.

→ The MUX selects either the output of register Y or a constant value 4 to be provided as input A of the ALU. The constant 4 is used to increment the contents of the PC. The two possible values of the MUX control input Select are Select4 and SelectY for selecting the constant 4 or register Y, respectively.

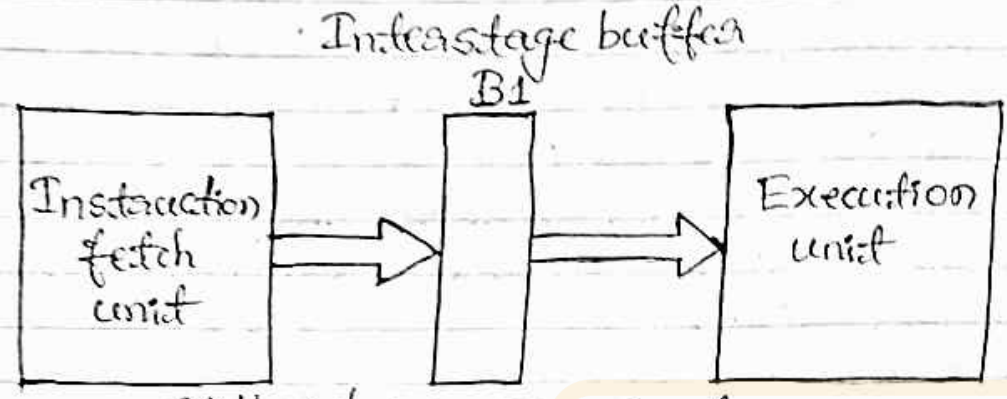
→ The data transfered from one register to another pass through the ALU and ALU performs some arithmetic or logic operation.
→ The instruction decoder and control logic unit is responsible for implementing the actions specified by the instruction loaded in the IR register. The decoder generates the control signals needed to select the registers involved and direct the transfer of data.
The registers, the ALU, and the interconnecting bus are collectively referred to as the datapath.

⑨.(b) Describe the basic idea of Instruction Pipeline.

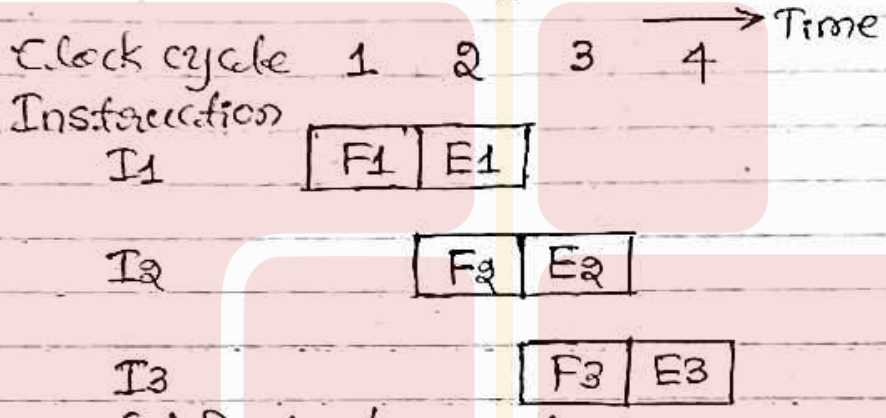
Ans:
Pipelining is a particularly effective way of organizing concurrent activity in a computer system.
The idea of pipelining can be used in a computer. The processor executes a program by fetching and executing instructions, one after the other. Let F_i and E_i refer to the fetch and execute steps for instruction I_i. Execution of a program consists of a sequence of fetch and execute steps, as shown in fig ⑨(b)(i).
Now consider a computer that has two separate hardware units, one for fetching instructions & another for executing them, as shown in fig ⑨(b)(ii). The instruction fetched by the fetch unit is deposited in an intermediate storage buffer, B1. This buffer is needed to enable the execution unit to execute the instruction while the fetch unit is fetching the next instruction. The results of execution are deposited in the destination location specified by the instruction. It is assumed that both the source and the destination of the data operated on by the instructions are inside the block labeled "Execution unit".



(i) Sequential execution



(ii) Hardware organization



(iii) Pipelined execution

Fig (9)(b) Basic idea of instruction pipelining

The computer is controlled by a clock which has a clock period. This period is such that the fetch and execute steps of any instruction can each be completed in one clock cycle. Operation of the computer proceeds as in fig (9)(b)(iii).

In the first clock cycle, the fetch unit fetches an instruction I1 (step F1) & stores it in buffer B1 at the end of the clock cycle.

In the second clock cycle, the instruction fetch unit proceeds with the fetch operation for instruction I2 (step F2). Meanwhile, the execution unit performs the operation specified by instruction I1 which is available to it in buffer B1 (step E1). By the end of the second clock cycle, the execution of instruction I1 is completed and instruction I2 is available.

Instruction I_2 is stored in B1, replacing I_1, which is no longer needed. Step E_2 is performed by the execution unit during the third clock cycle, while instruction I_3 is being fetched by the fetch unit. In this manner, both the fetch and execute units are kept busy all the time. If the pattern in fig (9)(b)(iii) can be sustained for a long time, the completion rate of instruction execution will be twice that achievable by the sequential operation given in fig (9)(b)(i).

The fetch and execute units in fig (9)(b)(ii) constitute a two-stage pipeline, in which each stage performs one step in processing an instruction. An inter-stage storage buffer, B1, is needed to hold the information being passed from one stage to the next. New type of information is loaded into this buffer at the end of each clock cycle.

(10)(a) Explain the process of Fetching Word from Memory in processor.

Ans:

To fetch a word of information from memory, the processor has to specify the address of the memory location where this information is stored and request a Read operation. The processor transfers the required address to the MAR, whose output is connected to the address lines of the memory bus. At the same time, the processor uses the control lines of the memory bus to indicate that a Read operation is needed. When the requested data are received from the memory they are stored in the register MDR, from where they can be transferred to other registers in the processor.

The connections for register MDR are illustrated in fig (10)(a)(i). It has four control signals: MDR_{in} and MDR_{out} control the connection to the internal bus, & MDR_{inE} and MDR_{outE} control the connection to the external bus. The circuit in fig (10)(a)(ii) will provide the additional connections.

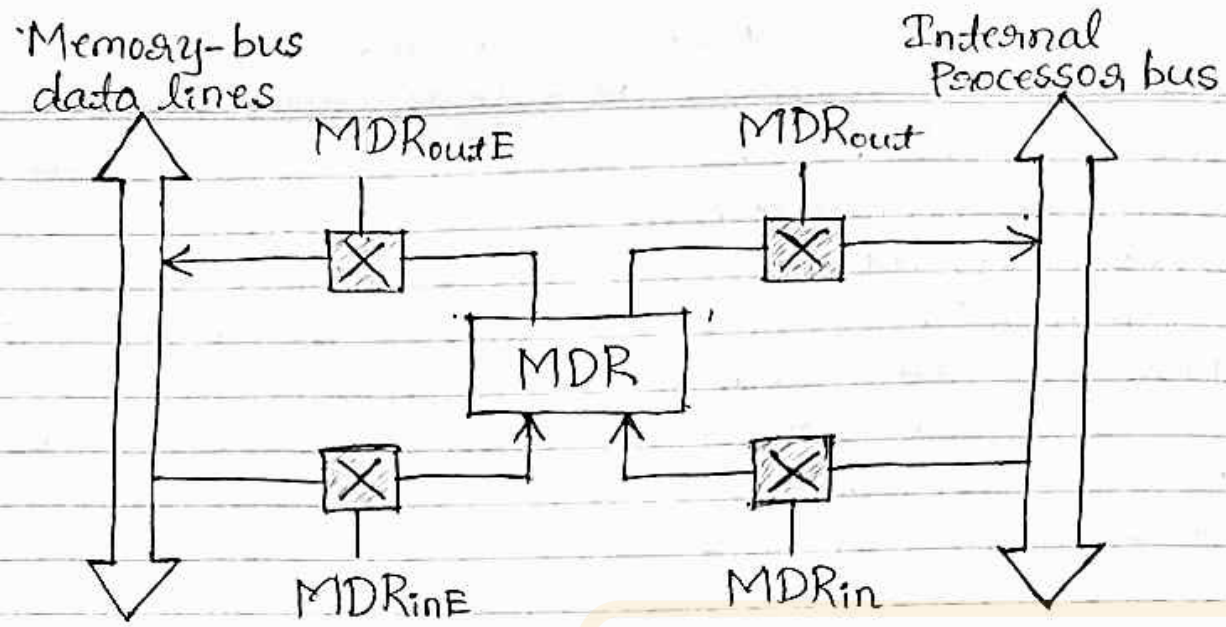


Fig (10)(a)(i) Connection and control signals for register MDR.

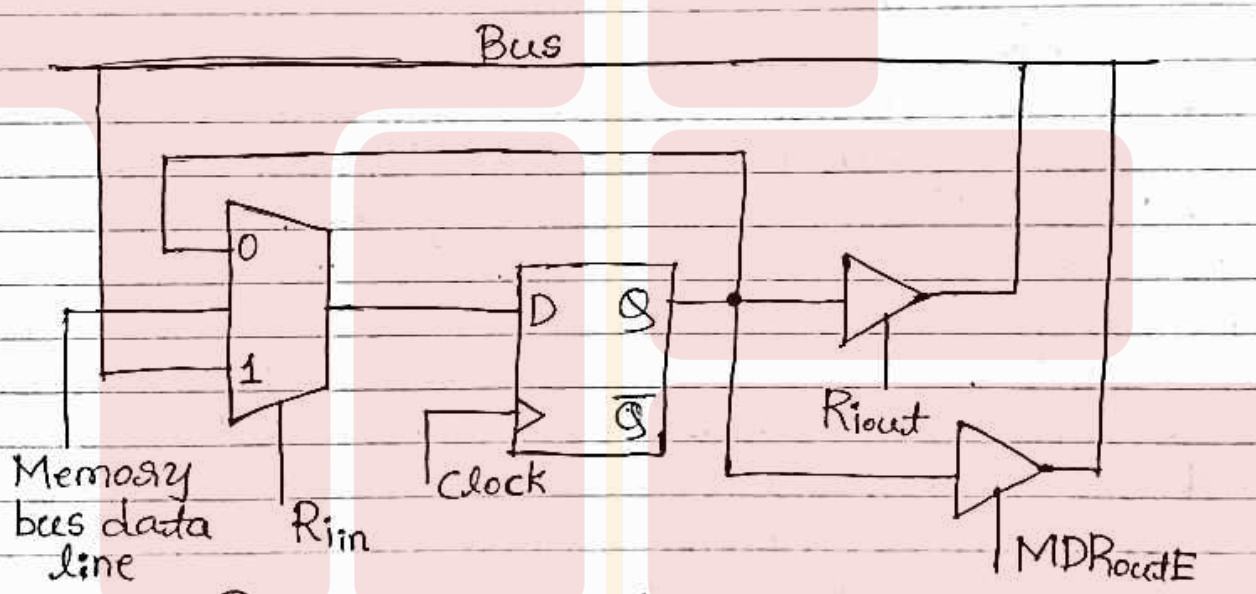


Fig (10)(a)(ii) Input and output gating for one register bit

- In the circuit of fig (10)(a)(ii), the memory bus data line is connected to the third input of the MUX, and this input is selected when $MDR_{inE} = 1$. A second tri-state gate controlled by MDRoutE can be used to connect the output of the flip flop to the memory bus.

During memory Read and Write operations, the timing of internal processor operations must be coordinated with the response of the addressed device on the memory bus. The processor completes

one internal data transfer in one clock cycle. The modern processors include a cache memory on the same chip as the processor. Typically, a cache could respond to a memory read request in one clock cycle. The processor waits until it receives an indication that the requested Read operation has been completed. Assuming that a control signal called Memory-Function-Completed (MFC) is used for this purpose. The addressed device sets this signal to 1 to indicate that the contents of the specified location have been read and are available on the data lines of the memory bus. As an example of a read operation, consider the instruction Move(R1), R2. The actions needed to execute this instruction are:

1. MAR ← [R1]
2. Start a Read operation on the memory bus
3. Wait for the MFC response from the memory
4. Load MDR from the memory bus
5. R2 ← [MDR]

The data received from the memory are loaded into MDR at the end of the clock cycle in which MFC signal is received. In the next clock cycle, MDR_{out} is activated to transfer the data to register R2. So, the memory read operation requires three steps, which can be described by the signals being activated as follows:

1. $R1_{out}$, MAR_{in}, Read
2. MDR_{inE}, WMFC
3. MDR_{out}, $R2_{in}$

where WMFC is the control signal that causes the processor's control circuitry to wait for the arrival of the MFC signal.

(10.) (b) Explain the Pipeline Performance of a Processor and pipeline stalls.

Ans:

The pipelined processor in a 4-stage pipeline completes the processing of one instruction in each clock cycle, which means that the rate of instruction processing is four times that of sequential operation. The potential increase in performance resulting from pipelining is proportional to the number of pipeline stages. This increase would be achieved only if pipelined operation of 4-stage pipeline could be sustained without interruption throughout program execution. Unfortunately, this is not the case.

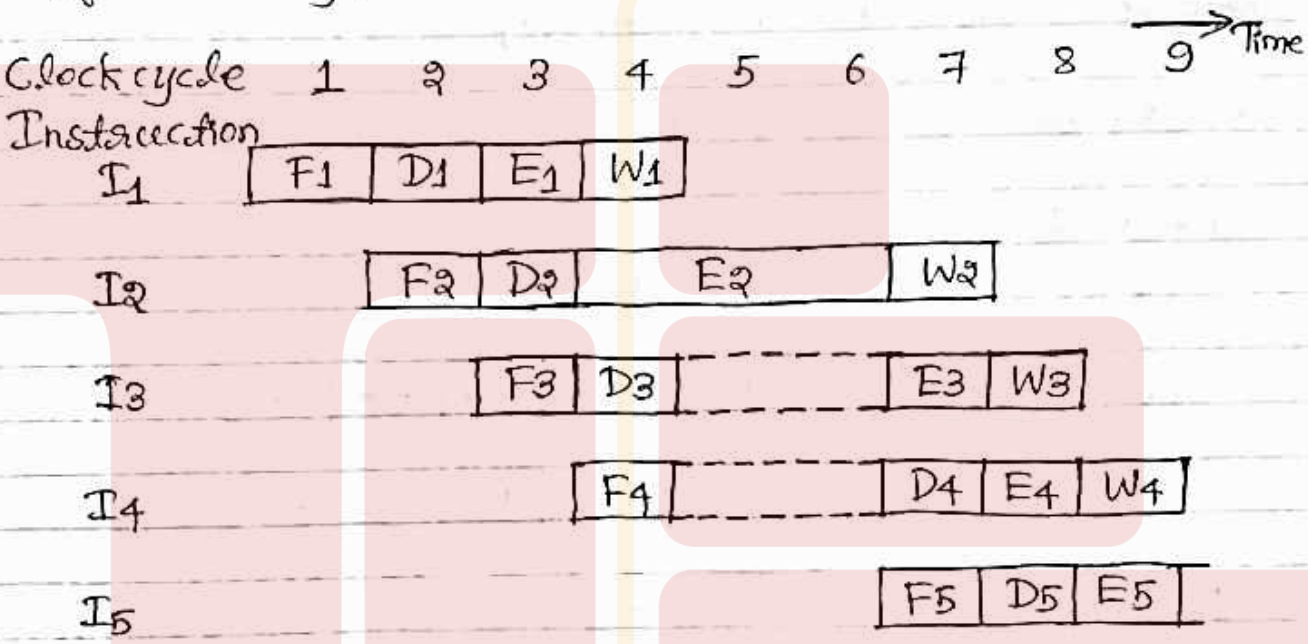


Fig (10)(b)(i) Effect of an execution operation taking more than one clock cycle

One of the pipeline stages may not be able to complete its processing task for a given instruction in the time alloted. In most operations, some operations, such as divide, may require more time to complete. Fig (10)(b)(i) shows an example in which the operation specified in instruction I2 requires three cycles to complete, from cycle 4 through cycle 6. So, in cycles 5 & 6, the Write stage do nothing because it has no data to work with. Meanwhile, the information in buffer B2 must remain intact until the Execute stage has completed its operation. This means that stage 2,

in turn, stage 1 are blocked from accepting new instructions because the information in B1 cannot be overwritten. So, steps D_4 and F_5 must be postponed as shown.

Pipeline operation in fig (10)(b)(i) is said to have been stalled for two clock cycles. Normal pipelined operation resumes in cycle 7. Any condition that causes the pipeline to stall is called a hazard. The current stage is an example of a data hazard. This is a condition in which either the source or the destination operands of an instruction are not available at the time expected in the pipeline.

(10)(c)(ii) Instruction execution steps in successive clock cycles.

Clock cycle	1	2	3	4	5	6	7	8	9 → Time
Stage									
F: Fetch	F1	F2	F2	F2	F2	F3			
D: Decode		D1	idle	idle	idle	D2	D3		
E: Execute			E1	idle	idle	idle	E2	E3	
W: Waite				W1	idle	idle	idle	W2	W3

(iii) Function performed by each processor stage in successive clock cycles

Fig (10)(c): Pipeline stall caused by a cache miss in F2.

The Pipeline may also be stalled because of a delay in the availability of an instruction. For example, this may be a result of a miss in the cache, requiring the instruction to be fetched from the main memory. Such hazards are often called control hazards or instruction hazards. The effect of a cache miss on pipelined operation is illustrated in fig (10)(c). Instruction I_1 is fetched from the cache in cycle 1, and its execution proceeds normally. However, the fetch operation for I_2, which is started in cycle 2, results in a cache miss. The instruction fetch unit must now suspend any further fetch requests and wait for I_2 to arrive. It is assumed that I_2 is received and loaded into buffer B1 at the end of cycle 5. The pipeline resumes its normal operation at that point.

An another representation of the operation of a pipeline in the case of a cache miss is shown in fig (10)(c)(iii). It gives the function performed by each pipeline stage in each clock cycle. The Decode unit is idle in cycles 3 through 5, Execute unit is idle in cycles 4 through 6, and the Write unit is idle in cycles 5 through 7. Such idle periods are called stalls, also called as bubbles in the pipeline. Once created as a result of a delay in one of the pipeline stages, a bubble moves downstream until it reaches the last unit.